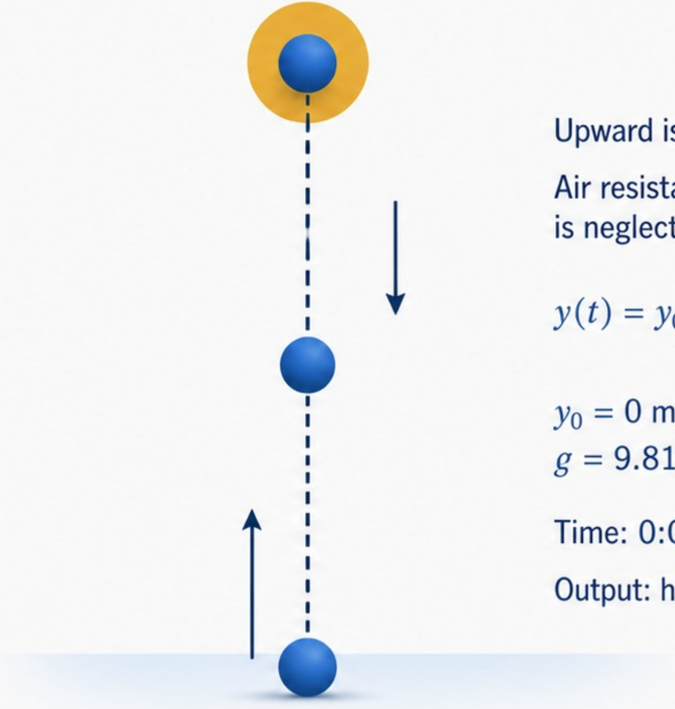
# Physics to Arrays and Plots

## From a familiar motion model to trustworthy computational evidence.

- A familiar motion model becomes computational evidence.
- Model –> arrays –> plot –> validation –> interpretation.



Upward is positive.

Air resistance is neglected.

$$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$$

$y_0 = 0$ m, $v_0 = 20$ m s$^{-1}$,
$g = 9.81$ m s$^{-2}$

Time: 0:0.1:4 s

Output: height $y$ versus time $t$

**1. Physical Model** → Understand and state the model.

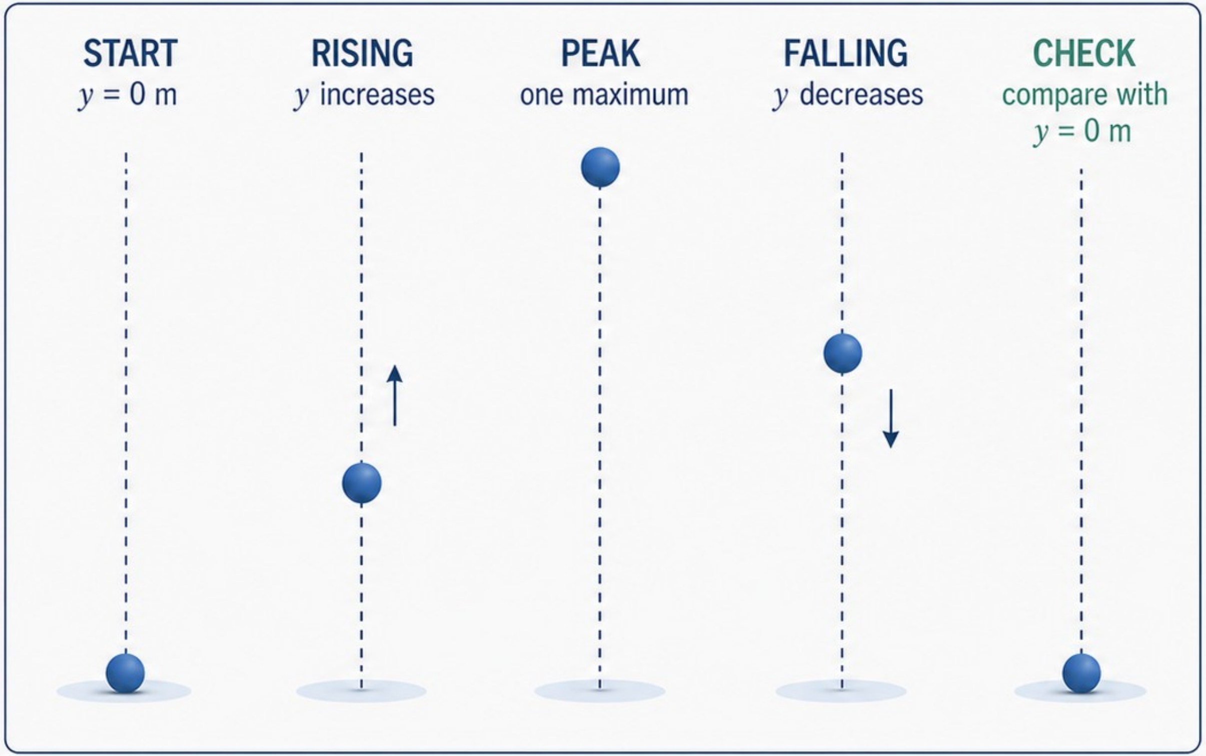
**2. Arrays** → Define variables and create arrays.

**3. Plot** → Visualize height versus time.

**4. Validation** → Check the boundary: $y(0) = 0 \text{ m} \Rightarrow y_m(1)$

**5. Interpretation** Explain the result in physical terms.

# Before MATLAB, sketch what must happen




Blue = motion | Green = expected check | Yellow = prompt

## Prediction before computation

- At $t = 0$ s, $y = 0$ m
- As $t$ increases, $y$ rises to one maximum
- After the peak, $y$ decreases
- Near the end, compare with $y = 0$ m

**Prediction first**

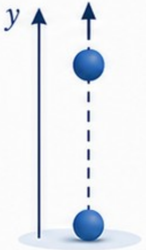
Sketch the shape before coding.

# Decide what the computer must remember

- Scalars store fixed values such as $y_0$, $v_0$, and $g$.
- An array stores many chosen times.
- Every stored quantity needs a meaning and unit.

**Physical model (vertical motion)**

$$y(t) = y_0 + v_0\, t - \frac{1}{2}\, g\, t^2$$



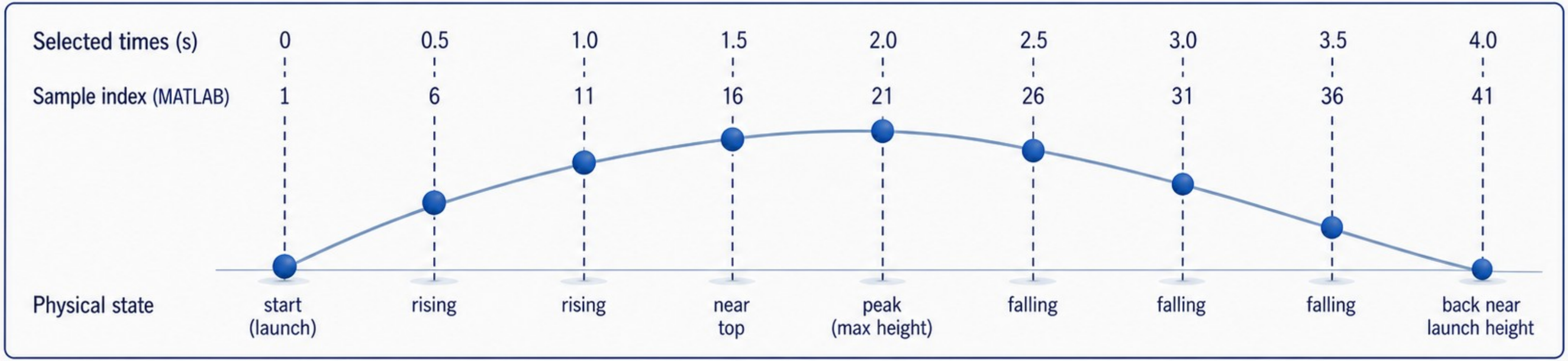

$y_0 = 0$ m
$v_0 = 20$ m s$^{-1}$
$g = 9.81$ m s$^{-2}$
$t = 0{:}0.1{:}4$ s

Upward is positive.
Air resistance is neglected.

| | Quantity | MATLAB name | Storage form | Meaning | Unit |
|---|---|---|---|---|---|
| SCALARS (fixed values) | $y_0$ | `y0_m` | scalar | initial height (launch height) | m |
| | $v_0$ | `v0_mps` | scalar | initial upward speed | m s$^{-1}$ |
| | $g$ | `g_mps2` | scalar | acceleration due to gravity (positive number) | m s$^{-2}$ |
| ARRAYS (many values) | $t$ | `t_s` | $1 \times N$ array | chosen time samples | s |
| | $y(t)$ | `y_m` | $1 \times N$ array | height at each time in $t$_s | m |

Blue = scalars (fixed values) | Green = arrays (many values) | Highlighted row = main output we will compute and plot

# Sampling is a choice, not the motion itself




### Array construction — Method 1

```
t_s = 0:0.1:4;
```

Endpoints: 0 s — 4 s

### Array construction — Method 2

```
t_s = linspace(0,4,41);
```

Endpoints: 0 s — 4 s

**41** samples

### Key points

- `t_s = 0:0.1:4` and `linspace(0,4,41)` make 41 time samples.
- The endpoints and count match.
- Sampling changes what we see, not the underlying equation.

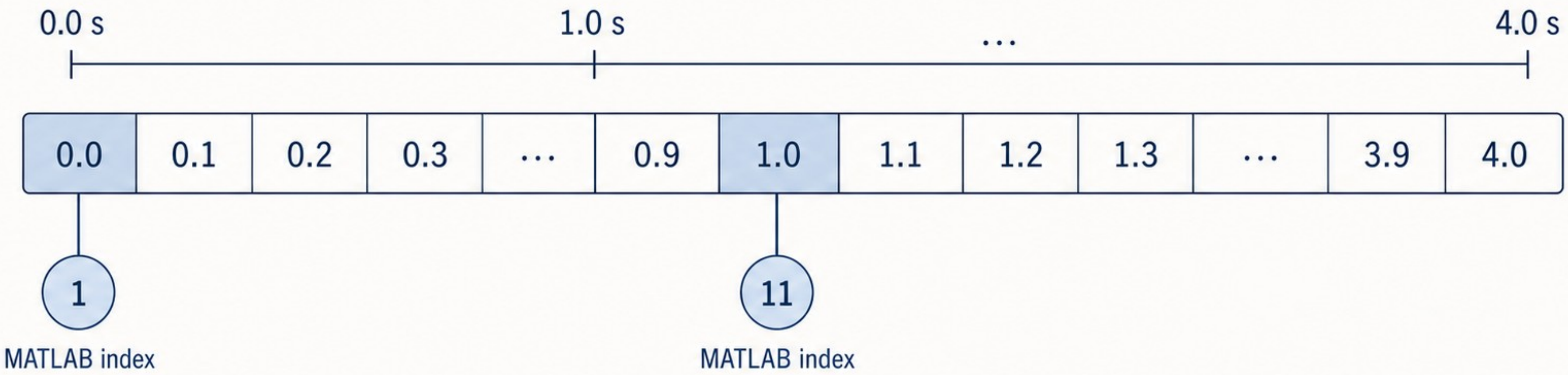
**Model (continuous idea):** $y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$

**Representation (samples we use):** selected moments from the same model

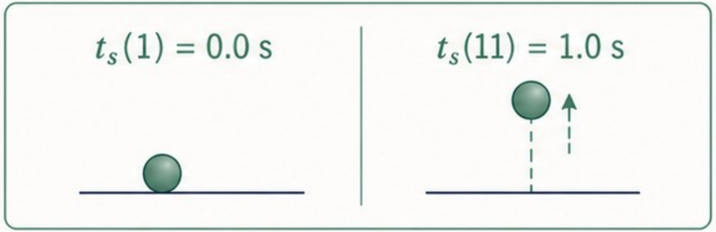
Different sampling → different detail, same underlying motion.

# One index represents one physical moment

**Time array:** $t_s = 0{:}0.1{:}4$ s




- $t_s(1)$ is the launch time.
- $t_s(11)$ selects 1.0 s.
- An index is a storage location, not a time value.




MATLAB indexing starts at 1

# Make one equation work for many times

$$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$$

starting position

$y_0 = 0$ m

launch contribution

$v_0 t \rightarrow$ metres

gravity contribution

$-\frac{1}{2} g t^2 \rightarrow$ metres

**One stored time → one height**

| | | | | |
|---|---|---|---|---|
| $t_s$ | [0.0 | 0.1 | 0.2 | ...] |
| $y_m$ | [0.0 | 1.95 | 3.80 | ...] |

Each stored time gets one height.

Every term on the right has units of metres.

| | | |
|---|---|---|
| $y_0$ m | $v_0 t$ m | $\frac{1}{2} g t^2$ m |

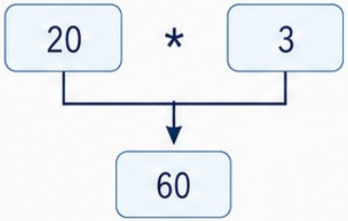
# The missing dot changes the operation

The dot means apply the operation element by element.

## Scalar multiplication

`20 * 3`

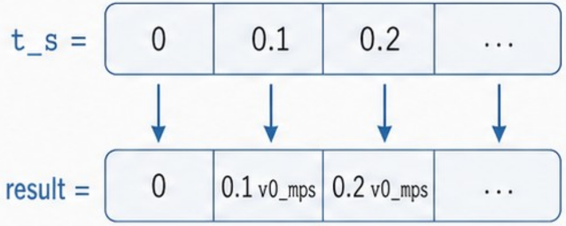
one scalar




`20 * 3` produces one scalar.

## One value per time

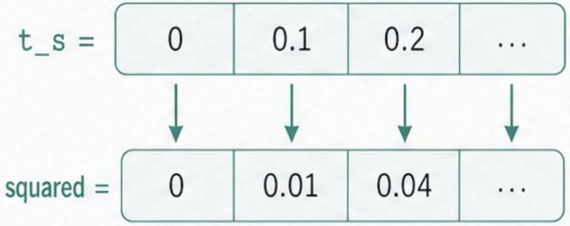
`v0_mps .* t_s`




`v0_mps .* t_s` produces one value per time.

## Square each stored time

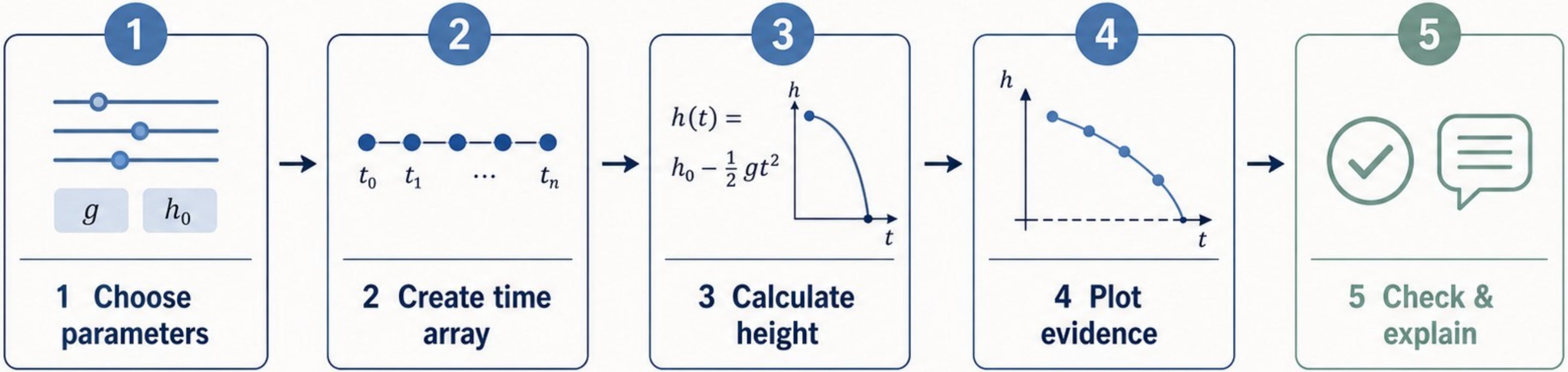
`t_s.^2`




`t_s.^2` squares every stored time value.

Checkpoint: Which expression returns an array? ✓ `v0_mps .* t_s` ✓ `t_s.^2`

# Build the algorithm before writing syntax




- Choose the physical parameters.
- Create the time array.
- Calculate height at every time.
- Plot, check, and explain.

# Read the Live Script in the order the physics runs

```
1  v0 = 20;
   g = 9.81;

2  t = linspace(0,4,11);

3  y = v0.*t - 0.5*g.*t.^2;

4  plot(t,y,'LineWidth',2);

5  y(1)
```

1. **PARAMETERS**
   Set the launch speed and gravity.
2. **ARRAY**
   Choose the sampled times.
3. **CALCULATION**
   Evaluate height for every time.
4. **PLOT**
   Turn the array into evidence.
5. **CHECK**
   At $t = 0$ s, $y$ must be 0 m.
   expected: $y(1) = 0$

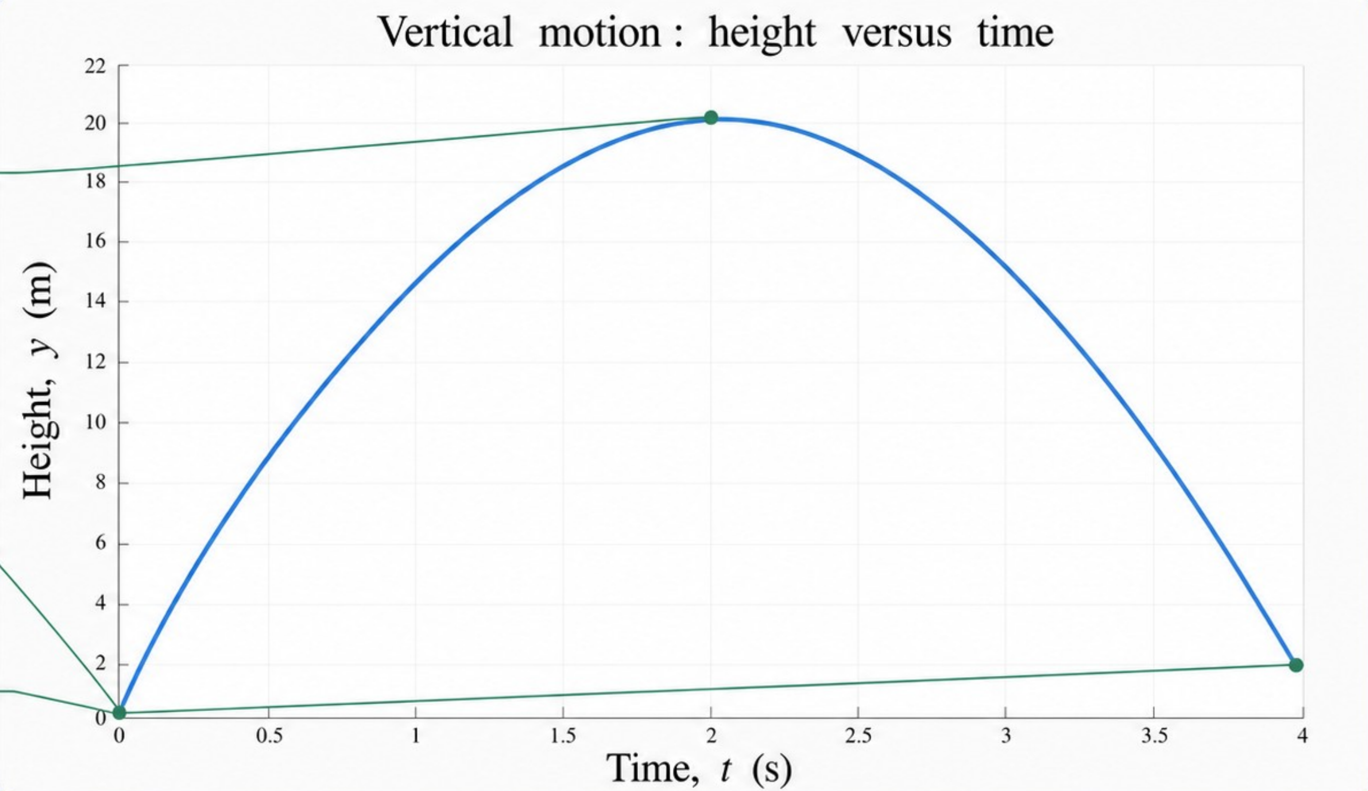
- Read comments and variable names first.
- Trace the array-ready model line.
- Locate the plot and boundary check.

# Compare the prediction with the plotted evidence

It rises to
one maximum.

The curve begins
at $y = 0$ m.

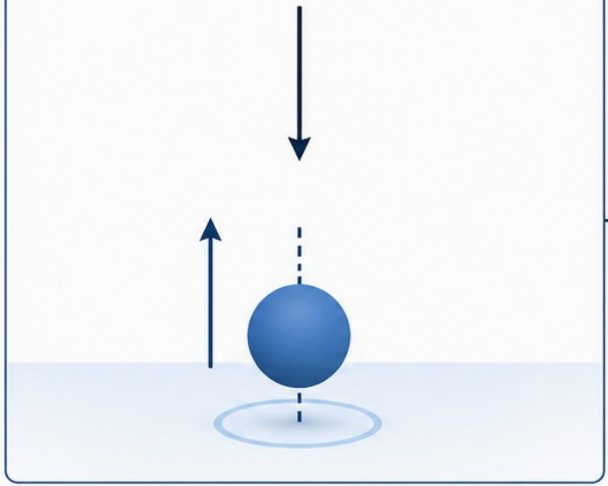
It falls toward
the launch height.




Labels and units make the evidence interpretable.

# Use a check the model must pass

**Known launch condition**



first stored value

$y_m(1)$

$t = 0\ \mathrm{s}$

$y(1) = 0.000\ \mathrm{m}$

At $t = 0$ s, the model must give $y = 0$ m.

**Executable boundary check**

```
assert(abs(y_m(1)-y0_m) < 1e-12)
```

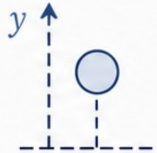
**PASS** — launch value matches $y0\_m$

- The known condition is $y(0) = 0$ m.
- In a one-based array this is $y_m(1)$.

A smooth graph is not enough: check a known condition.

# State what the computation lets us claim

### MODEL




$$y(t) = y_0 + v_0 t - \frac{1}{2} g t^2$$

upward positive;
air resistance neglected

→

### SAMPLING

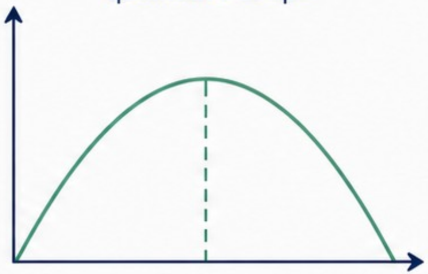
time array: 0:0.1:4 s

sampled positions

→

### GRAPH

qualitative shape



rises to one maximum, then falls

→

### CHECK

$$y_m(1) = 0 \text{ m}$$

starting boundary condition

Under these assumptions, the ball rises to one maximum and the boundary check supports $y(0) = 0$ m.

Not supported: the model proves real motion exactly.

A result needs a unit, a trend, and a limitation.

# Week 1 exit ticket

1. Name the output variable and its unit.

2. Explain what $t_s(11)$ selects.

3. Identify the operator that squares every value in $t_s$.

4. State one graph prediction and one value that can be validated.

Make each computational step traceable.